MGS613 Project Final Report

AIRBNB – Data Analysis

1. Introduction

"Belong anywhere." - Airbnb

Founded in 2007, Airbnb has revolutionized the way that people travel by allowing millions of guests to connect with unique accommodations all over the world. First starting as a small startup with short-term rentals, Airbnb currently spans across more than 220 countries and regions, with listings in over 100,000 cities and towns. From busy urban apartments to secluded rural getaways, the platform provides variety for travelers while giving hosts the opportunity to share their space and culture in substantial ways. To date, these hosts have earned well over $250 billion, illustrating just how big an economic impact the platform can have.

With such a wide reach, the data footprint of Airbnb is huge, carrying valuable insights on travel trends, guest preferences, and global market dynamics. The scale at which the data needs to be managed calls for a sophisticated database structure that captures patterns for enhancing guest experiences, improving engagement with hosts, and increasing operational efficiency. Analyzing booking patterns, user demographics, and demand in various regions can help Airbnb optimize their offerings and enhance their decision-making process continuously.

Our project will design a comprehensive database model simulating key components in the Airbnb structure, focusing on these critical data points. Using SQL and advanced data modeling, our team will construct an interrelated set of tables to analyze Airbnb's business performance and provide insights on customer trends across regions

2. Data

The dataset for this project is chosen from the kaggle. It contains information about the house, owner, country, reviews, booking details. The entire reference dataset contains more than 10 tables and 8 entities, from which we picked up only few scenarios and important use cases which includes four entities namely Owner, Booking, Property and Customer and three relationships.

Business rules:

• Each customer must have a unique Customer ID.

• Customers must provide their first name, last name, email, phone number, and address to create an account.

• A customer can create multiple bookings, but each booking must be associated with a unique customer.

• Each booking must have a unique Booking ID.

• A booking must have a Check-In Date and Check-Out Date, with the check-in date being earlier than the check-out date.

• A booking must have a defined Booking Status (e.g., Confirmed, Pending, Canceled).

• The Total Booking Price should be calculated based on the property's Price Per Night and the total number of nights.

• Each booking is associated with exactly one property and one customer.

• Each property must have a unique Property ID.

• Properties must have details such as Property Type (e.g., Apartment, House), Address, City, State, and Country.

• A property can have multiple bookings associated with it, but overlapping dates for active bookings are not allowed.

• Each property is owned by exactly one owner, but an owner can own multiple properties.

• Each owner must have a unique Owner ID.

• Owners must provide their name, email, and phone number to register as property owners.

• An owner can own multiple properties, but each property must be linked to exactly one owner.

Source files and documentation:

- Reference Dataset: https://www.kaggle.com/datasets/arianazmoudeh/airbnbopendata/data & Chat-GPT
- Dataset To Be Implemented: Database is manipulated using AI based on the above reference dataset.

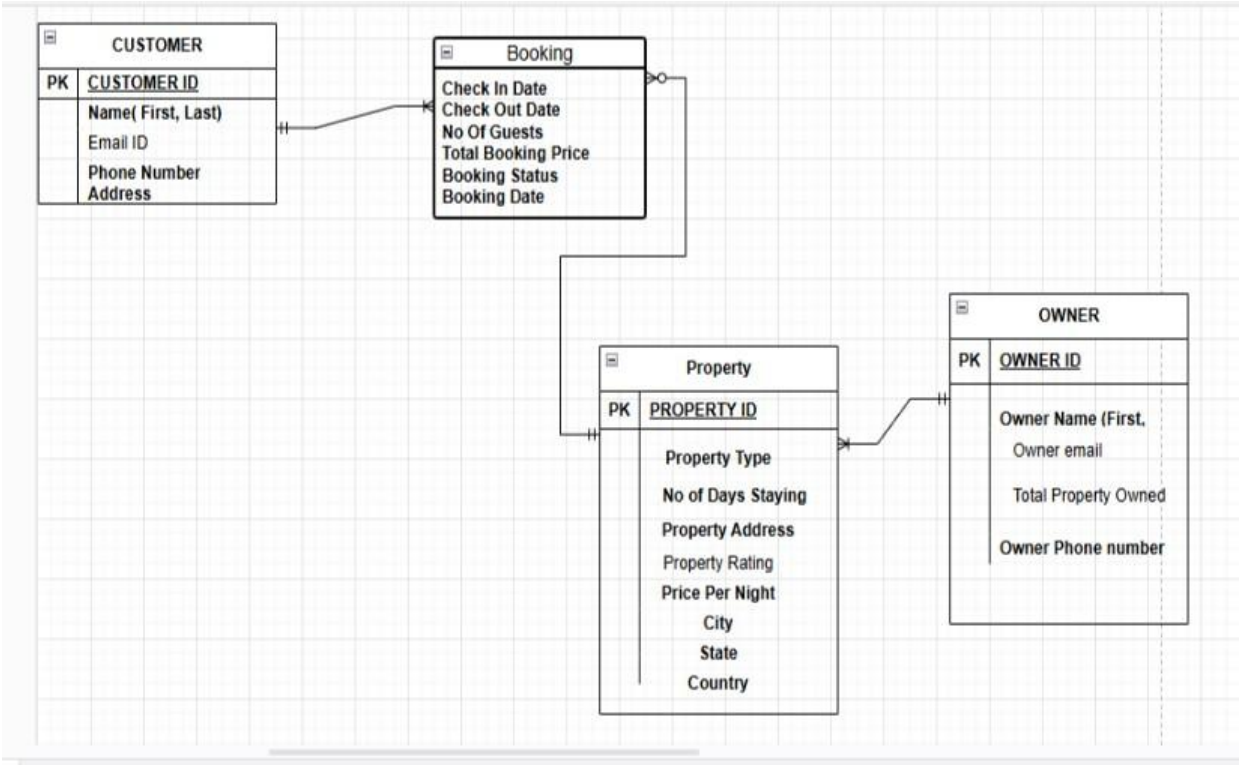
Data Dictionary:

Column Name	Data Type	Description
Customer ID	Numeric	Unique ID for Customer Database
Customer First Name	Text	Customer First Name
Customer Last Name	Text	Customer Last Name
Email ID	Text	Customers Email ID for communication and confirmation

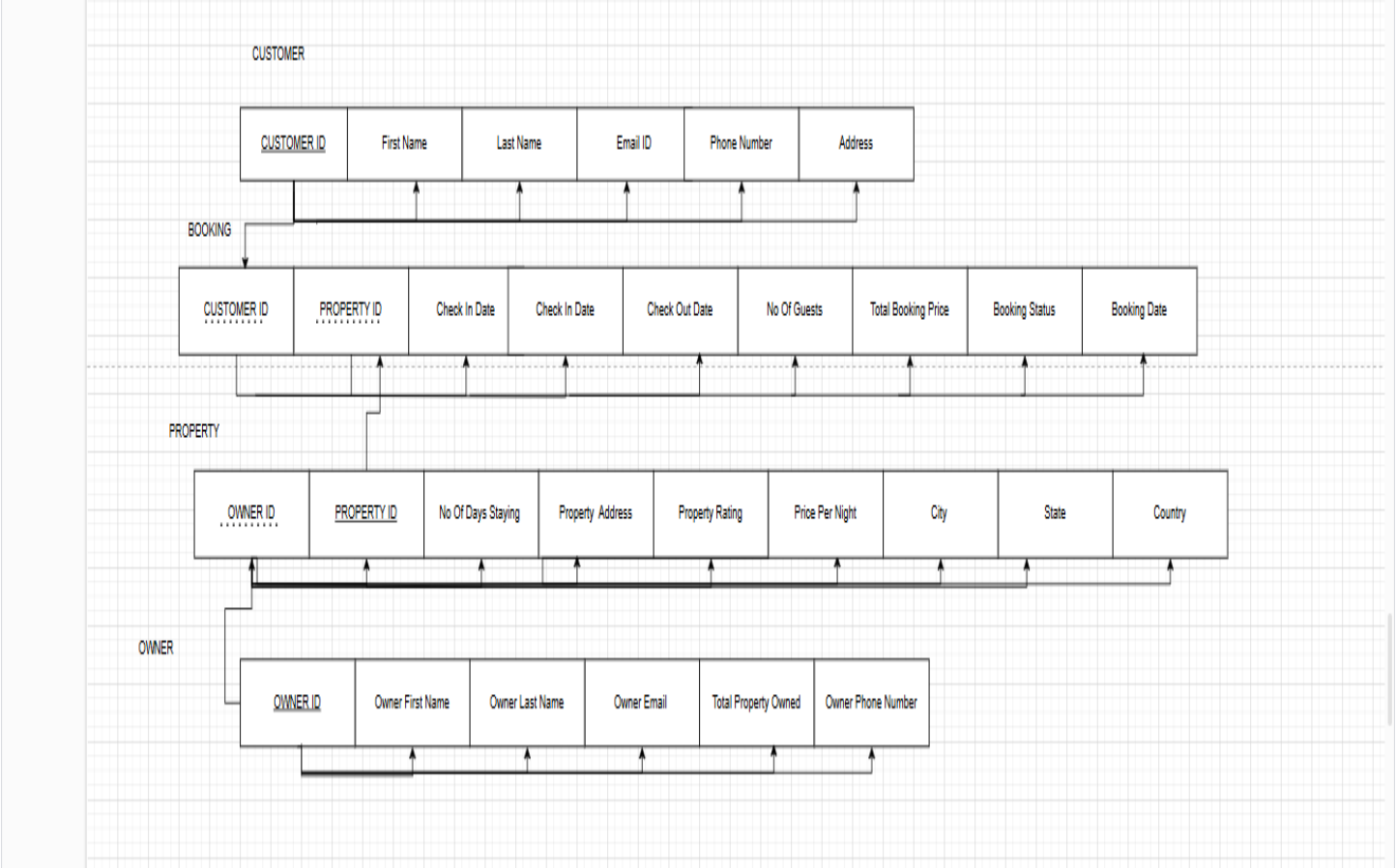
Phone Number	Numeric	Customers Phone Number
Address	Text	Customers Home address
Check-in date	Date	Booking start date of the customer
Check-out date	Date	Booking last date of the customer
No of guests	Numeric	Number of guests for the booking
Total Booking Price	Numeric	Customer Total Bill
Booking status	Text	Booked or Available
Booking Date	Date	The date on which the booking is done
Owner ID	Numeric	Unique ID for Owner Database
Owner email	Text	Owner email ID for communication and confirmation
Total properties owned	Numeric	Total no of properties owned by the owner
Owner First Name	Text	Owner First Name
Owner Last Name	Text	Owner Last Name
Owner phone number	Numeric	Owner Phone number
Property Address	Text	Owner Property address
Price per night	Numeric	Property price per night
City	Text	City in which the property is located
State	Text	State in which the property is located
Country	Text	The country in which the property is located.
Property type	Text	Whether it is a Room or the entire house

No of days staying	Numeric	Number of guests staying
Property ID	Numeric	Unique ID for Property Database
Property Rating	Numeric	Out of 5 which rating is allocated to the property 1 being the lowest and 5 being th highest

ER Diagram:



Relational Schema:



3. Database Implementation

The Airbnb dataset was carefully designed and structured to maintain data integrity and usability. Before implementation, extensive **data cleaning** was performed to ensure the quality of data. For instance, all phone numbers were validated as 10-digit numbers with the +1 extension, while all IDs such as Customer ID, Owner ID, and Property ID were verified as unique. Null values in critical fields like Customer Name, Email ID, and Booking Status were eliminated, ensuring a seamless implementation.

The database is organized into several interrelated tables, with the primary ones being **Customer**, **Booking**, **Property**, and **Owner**. Each table is uniquely identifiable by its respective primary key: CustomerID for customers, OwnerID for owners, PropertyID for properties, and BookingID for bookings. This structured approach ensures consistency and ease of access for the dataset.

The relationships between the tables are designed to reflect real-world scenarios. For example, each customer can make multiple bookings, but a single booking is associated with only one customer. Similarly, each property belongs to a single owner, but an owner can manage multiple properties.

These relationships are maintained using foreign keys, such as CustomerID in the Booking table referencing the Customer table and PropertyID in the Booking table referencing the Property table.

Each table is populated with relevant attributes. For the **Customer Table**, key attributes include CustomerID, FirstName, LastName, EmailID, PhoneNumber, and Address. The **Owner Table** stores information such as OwnerID, FirstName, LastName, EmailID, PhoneNumber, and the total number of properties owned. The **Property Table** includes PropertyID, PropertyAddress, PricePerNight, City, State, PropertyType, and PropertyRating, while the **Booking Table** tracks transactional data like CheckInDate, CheckOutDate, NumberOfGuests, TotalBookingPrice, BookingStatus, and BookingDate.

The database schema follows a relational model. Each primary table is uniquely identified by its primary key, while foreign keys establish the relationships between tables. For instance, the Booking table links customers and properties through the CustomerID and PropertyID foreign keys. These relationships allow for advanced analytics and ensure the database adheres to relational principles.

This implementation supports a wide range of analytics and insights. For example, revenue generated by each owner, and booking trends over time. The dataset is designed to facilitate such analyses without redundancy or inconsistency.

To illustrate, here is an example of the structured data: A customer named John Doe, identified by CustomerID 102345, makes a booking for a property located at 456 Oak Avenue, Chicago, IL, owned by Alice Brown (OwnerID 204678). The property, rated 4.7, has a price of $150 per night. John books the property for five nights from December 20 to December 25, 2024, for two guests, with a total booking price of $750. The booking is confirmed and recorded in the Booking table with all relevant details.

In summary, this Airbnb dataset was implemented with a focus on data quality, relational consistency, and usability for analysis and management. The structured tables, clear relationships, and well-defined attributes ensure the database is robust and ready for both operational and analytical use cases.

DDL and Insert Queries:

CSV INSERTION:

We uploaded the CSV file using the header mapping and unmapping feature of the Load Data option in Workshop -> Utilities -> Data Workshop.

Entities	**SQL Queries(Create and Insert Queries)**
Customer	CREATE TABLE CUSTOMER (CustomerID NUMBER PRIMARY KEY NOT NULL, CustomerFirstName VARCHAR2(50) NOT NULL, CustomerLastName VARCHAR2(50) NOT NULL, EmailID VARCHAR2(100) NOT NULL, PhoneNumber VARCHAR2(15) NOT NULL, Address VARCHAR2(255) NOT NULL); INSERT INTO OWNER (OWNERID,OwnerFIRSTNAME,OwnerLASTNAME,EMAIL,TotalNoofPropertiesOwned,OwnerPHONENUMBER) VALUES (1,'John','Doe','johndoe@yahoo.com',3, '+1234567890');
Owner	CREATE TABLE Owner (OwnerID INT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY, OwnerFirstName VARCHAR2(100) NOT NULL, OwnerLastName VARCHAR2(50) NOT NULL, OwnerPhoneNumber VARCHAR2(15) NOT NULL, TotalNoOfPropertiesOwned VARCHAR2(15), OwnerEmailID VARCHAR2(100) NOT NULL);

	INSERT INTO CUSTOMER (CUSTOMERID, FIRSTNAME, LASTNAME, EMAIL, PHONENUMBER, ADDRESS) VALUES (2, 'Jane', 'Smith', 'jane.smith@yahoo.com', '987-654-3210', '456 Elm St, Buffalo, New York');
Property	CREATE TABLE Property (PropertyID INT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY, OwnerID INT, PropertyName VARCHAR2(100), PropertyType VARCHAR2(100) NOT NULL, NoOfDaysStaying NUMBER, NoOfGuests NUMBER, Country VARCHAR2(100), City VARCHAR2(50), State VARCHAR2(100), PropertyRating NUMBER, PropertyAddress VARCHAR2(225), PricePerNight NUMBER(10, 2) NOT NULL, FOREIGN KEY (OwnerID) REFERENCES Owner(OwnerID)); INSERT INTO PROPERTY (PROPERTYID, PROPERTYTYPE, NOOFDAYSSTAYING, NOOFGUESTS, COUNTRY, CITY, STATE, PROPERTYRATING) VALUES (101, 'Apartment', 5, 3, 'USA', 'New York', 'NY', 4.5);
Booking	CREATE TABLE BOOKING (BOOKINGID INT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,

```
    CheckInDate DATE NOT NULL,

    CheckOutDate DATE NOT NULL,

    NoofGuests NUMBER NOT NULL,

    TotalBookingPrice NUMBER(10, 2) DEFAULT 0.00,

    BookingStatus VARCHAR2(50) DEFAULT 'Pending',

    BookingDate DATE DEFAULT SYSDATE,

    PropertyID NUMBER NOT NULL,

    CustomerID NUMBER NOT NULL,

    CONSTRAINT FK_PROPERTY FOREIGN KEY (PropertyID) REFERENCES
PROPERTY(PropertyID) ON DELETE CASCADE,

    CONSTRAINT FK_CUSTOMER FOREIGN KEY (CustomerID)
REFERENCES CUSTOMER(CustomerID) ON DELETE CASCADE);

INSERT INTO BOOKING
(BOOKINGID,PROPERTYID,CUSTOMERID,CHECKINDATE,CHECKOUTDA
TE,NOOFGUESTS,TOTALBOOKINGPRICE,BOOKINGSTATUS,BOOKINGD
ATE) VALUES (1,101, 2,TO_DATE('2024-12-20', 'YYYY-MM-
DD'),TO_DATE('2024-12-25', 'YYYY-MM-DD'),2,1500.50,
'Confirmed', SYSDATE );

UPDATE INTO BOOKING

SET

    Payment_Status = CASE

                WHEN BookingStatus = 'Confirmed' THEN 'Paid'

                WHEN BookingStatus = 'Canceled' THEN 'Refunded'

                WHEN BookingStatus = 'Pending' THEN 'Pending'

                ELSE Payment_Status

                END

WHERE
```

	BookingStatus IN ('Confirmed', 'Canceled', 'Pending');

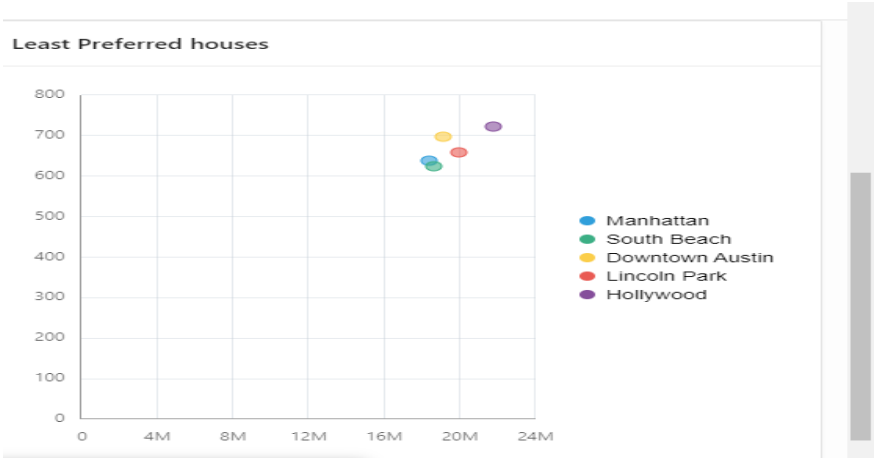
The following are link for the source code and csv file:
https://drive.google.com/drive/folders/1jx0lK5S5Lyc0uj-SajQjP3lmB-7WI63_?usp=sharing

4. Analysis

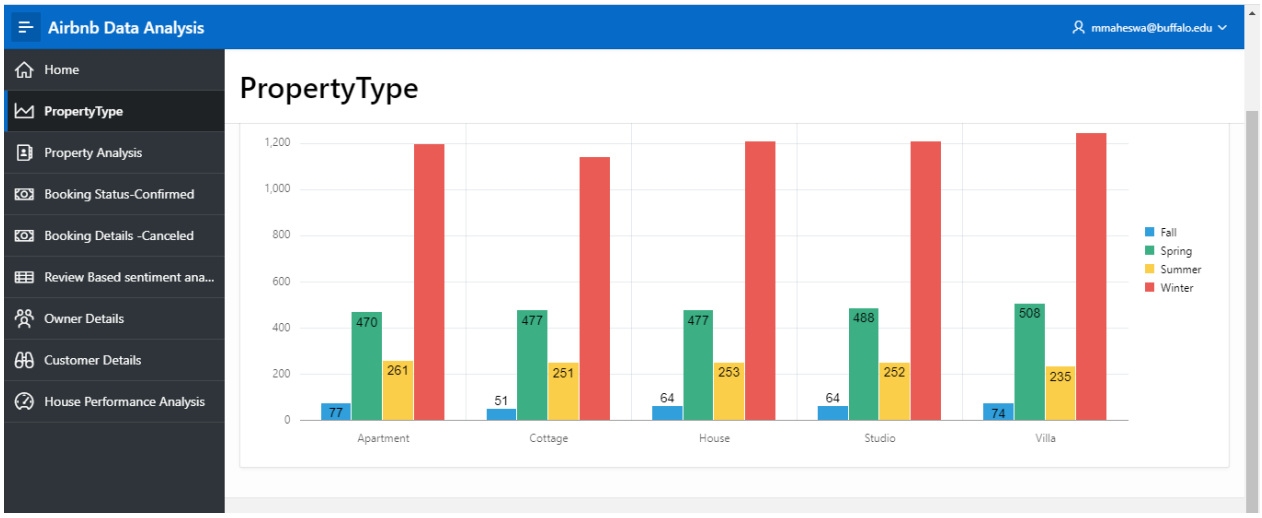
Proposed questions:

1. House Performance analytics:
   The least number of houses booked and less revenue generating house.

```
SELECT
p.Neighbourhood AS "Neighbourhood",
COUNT(b.BookingID) AS "TotalBookings",
SUM(b.TotalBookingPrice) AS "TotalRevenue"
FROM
Property p
JOIN
Booking b ON p.PropertyID = b.PropertyID
WHERE
b.BookingStatus = 'Confirmed'
GROUP BY
p.Neighbourhood
HAVING
COUNT(b.BookingID) > 0
ORDER BY
"TotalRevenue" ASC, "TotalBookings" ASC;
```

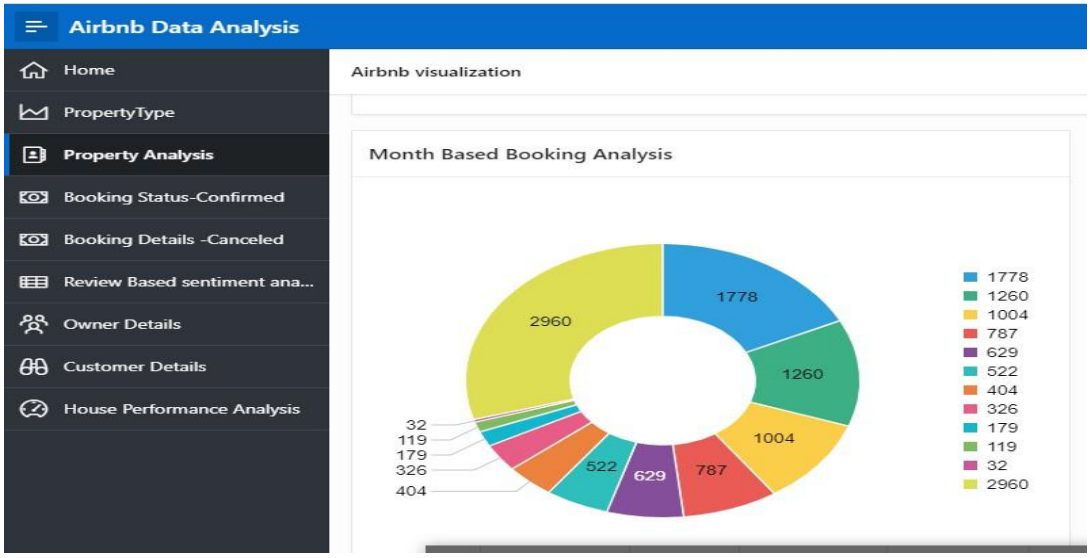


Most preferred property type based on season



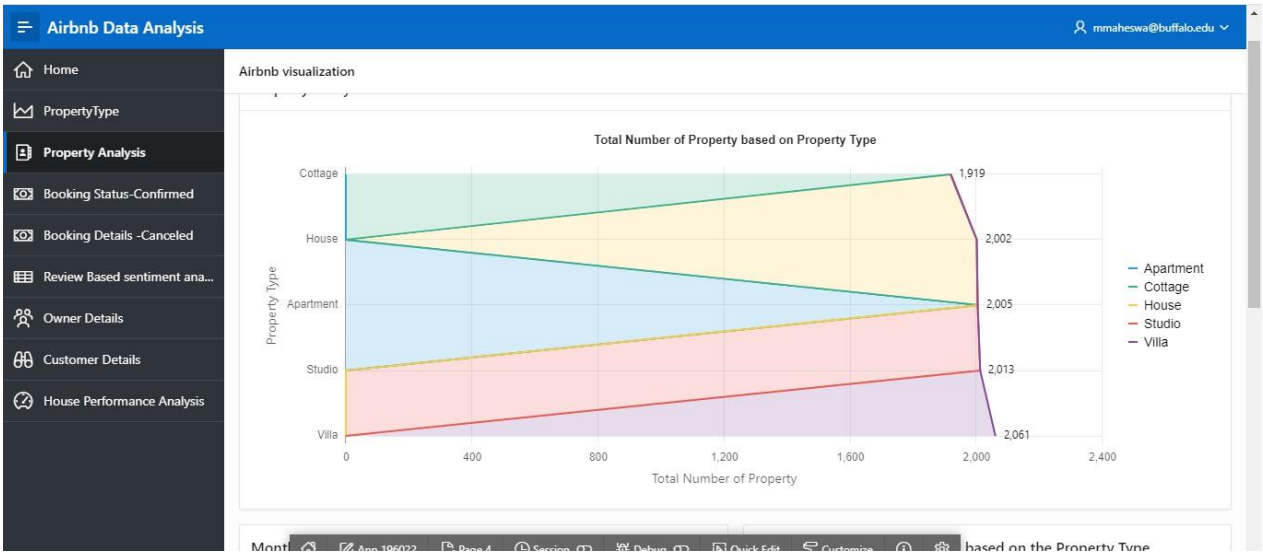
Seasonal trends which months houses are mostly booked.

```
SELECT
    TO_CHAR(BookingDate, 'Month') AS "Month",
    COUNT(*) AS "Bookings"
FROM
    Booking
GROUP BY
    TO_CHAR(BookingDate, 'Month')
ORDER BY
    TO_DATE("Month", 'Month');
```



Most preferred property Type.

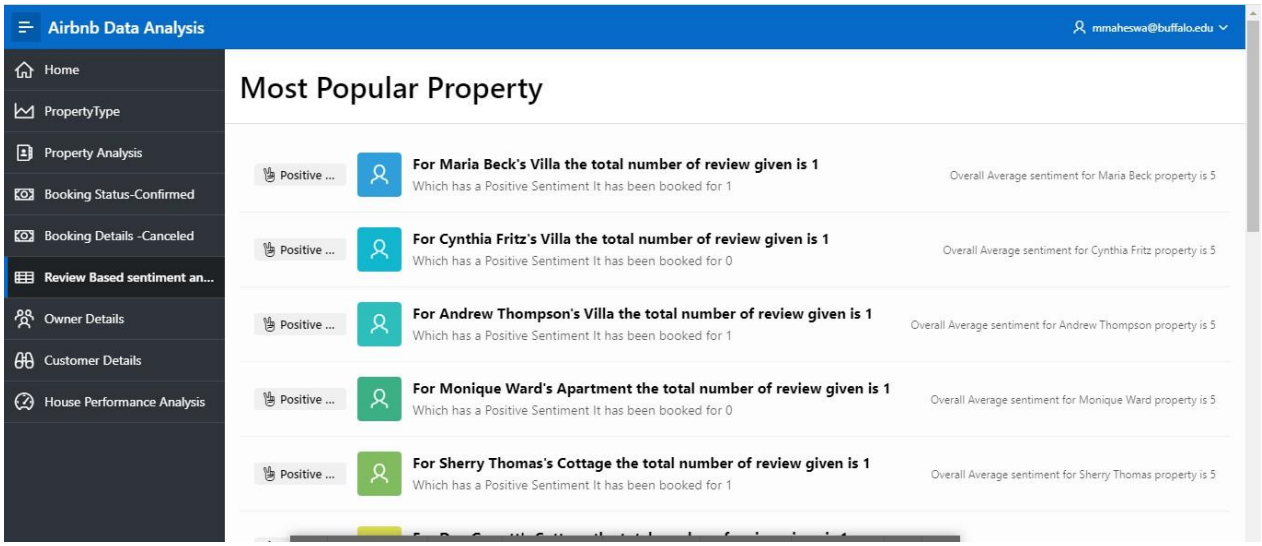
```
SELECT
    PropertyType AS "Category",
    COUNT(*) AS "Count"
FROM
    Property
GROUP BY
    PropertyType
ORDER BY
    PropertyType;
```



2. Owner's business performance analytics:
   Review sentiment analysis for the houses held by the owner, which are the popular house types.

```
SELECT
    o.OwnerFirstName || ' ' || o.OwnerLastName AS "OwnerName",
    p.PropertyType AS "PropertyType",
    COUNT(p.PropertyRating) AS "TotalReviews",
    SUM(CASE WHEN b.BookingStatus = 'Confirmed' THEN 1 ELSE 0 END) AS "BookingStatus",
    ROUND(AVG(p.PropertyRating),2) AS "AverageSentiment",
    CASE
        WHEN AVG(p.PropertyRating) >= 4 THEN 'Positive Sentiment'
        WHEN AVG(p.PropertyRating) BETWEEN 2 AND 3.99 THEN 'Neutral Sentiment'
        ELSE 'Negative Sentiment'
```

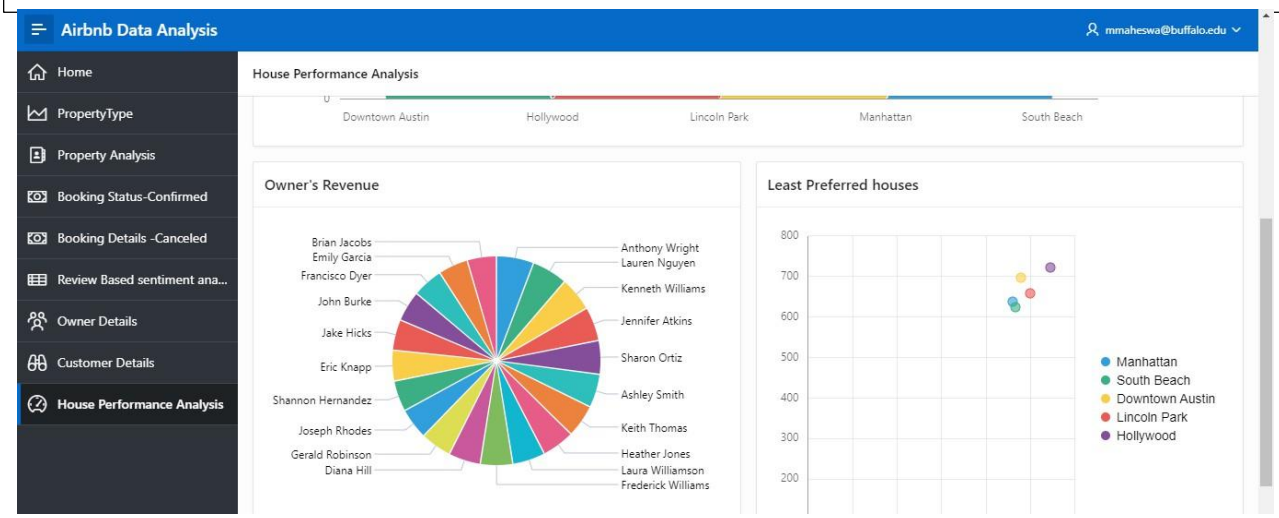
```
    END AS "SentimentCategory"
FROM
    Owner o
JOIN
    Property p ON o.OwnerID = p.OwnerID
JOIN
    Booking b ON p.PropertyID = b.PropertyID
GROUP BY
    o.OwnerFirstName, o.OwnerLastName, p.PropertyType
ORDER BY
     "AverageSentiment" DESC;
```



Total turnover revenue of the owner.

```
SELECT
    o.OwnerID,
    o.OwnerFirstName || ' ' || o.OwnerLastName AS "OwnerName",
    SUM(b.TotalBookingPrice) AS "TotalTurnoverRevenue"
FROM
    Owner o
```

```
JOIN
   Property p ON o.OwnerID = p.OwnerID
JOIN
   Booking b ON p.PropertyID = b.PropertyID
WHERE
   b.BookingStatus = 'Confirmed'
GROUP BY
   o.OwnerID, o.OwnerFirstName, o.OwnerLastName
ORDER BY
   "TotalTurnoverRevenue"  DESC;
```



Neighborhood based revenue comparison.

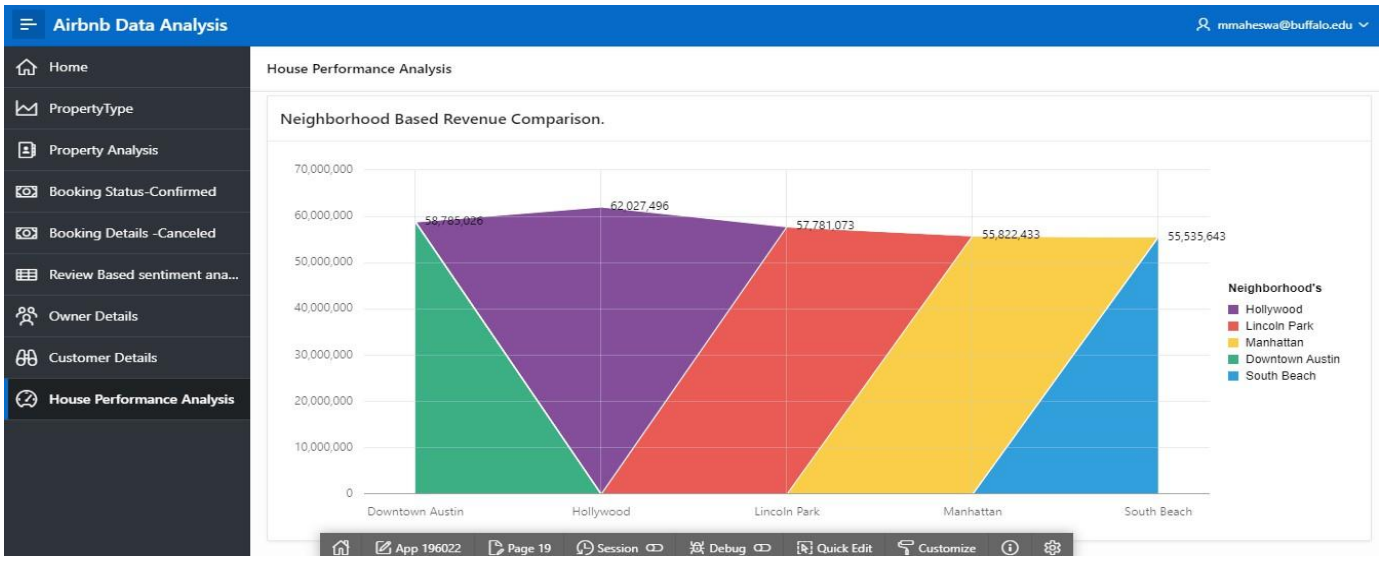
```
SELECT

   p.neighbourhood AS "neighbourhood",

   SUM(b.totalBookingPrice) AS "totalBookingPrice"

FROM  Booking b JOIN  Property p ON  b.propertyId = p.propertyId

GROUP BY p.neighbourhood;
```



References:

About us - Airbnb Newsroom